5 Analyse descendante, analyse LL

Intuition

Dans l'analyse descendante, la difficulté consiste à prédire la bonne règle de grammaire au bon moment (à trouver le bon chemin dans l'automate). Il serait intéressant de pouvoir construire un processus automatique permettant cette prédiction, cela permettrait de connaître à tout moment le chemin à parcourir.
Afin de construire un tel oracle, nous allons essayer de nous baser sur le (ou les) symboles en début du mot restant à analyser. Pour cela, il nous faut nous tourner vers la grammaire pour calculer quels sont les symboles permettant de décider la prédiction correcte pour ré-écrire le symbole en sommet de pile.
Hélas, il apparaît rapidement que cela n'est pas toujours possible dans le cadre général. Ainsi, on voit rapidement dans la trace d'analyse descendante de l'exemple E4.19 qu'un «oracle» devrait prédire la règle 1 au premier appel et la règle 2 au second, alors que l'état du système est le même dans les deux cas (on est au même état, même mot à analyser et même symbole en sommet de pile). Il apparaît sur cet exemple qu'une grammaire ayant un symbole récursif à gauche posera problème pour un tel oracle.
Nous verrons que ce ne sera pas le seul problème qui se posera pour une prédiction déterministe, mais commençons par voir ce que nous pouvons faire dans ce cas.

1 Élimination de la récursivité à gauche

Définition 1.1 Une grammaire est dite **récursive à gauche** si il existe un non terminal $A \in N$, tel que $A \vdash^{+} A\alpha$.

Dans l'exemple E4.19, la règle $E \to E + T$ est une règle récursive à gauche (E est dans la partie gauche et à gauche de la partie droite). Attention, une grammaire peut être récursive à gauche sans qu'aucune production ne soit directement récursive à gauche. Ainsi, la grammaire G :

$A \to Ba|\varepsilon \quad B \to Ab|b$

est récursive à gauche car $A \vdash Ba \vdash Aba$.

Théorème 1.1 Toute grammaire hors-contexte G admet une grammaire faiblement équivalente sans récursivité à gauche.

La démonstration du théorème tient dans la transformation de la grammaire.

1.1 Cas de récursivité directe

On s'intéresse au cas où la grammaire contient une production récursive à gauche (de la forme $A \to A\alpha$).
Dans ce cas, l'ensemble des A-productions peut se mettre sous la forme générale :

$$A \to A\alpha \mid \beta \text{ avec } \beta \text{ ne commençant pas par A}$$

Ces A-productions peuvent être remplacées par les productions :

$A \to \beta A'$
$A' \to \alpha A'|\varepsilon$

où A' est un nouveau non-terminal.

1.2 Cas de la récursivité indirecte

Dans ce cas, on peut appliquer un pivot de Gauss pour supprimer cette récursivité indirecte. Ainsi, on va ordonner les non-terminaux A_1, A_2, ..., A_n et les traiter l'un après l'autre.

Suppression de la récursivité gauche

Entrée:
$G = (N, T, P, S)$; // une grammaire hors-contexte propre
Sortie:
$G' = (N, T, P', S)$; // équivalente à G et sans récursivité gauche.
Réalisation:
// $N = \{A_1, ..., A_n\}$
pour chaque $A_i \in N$
On transforme G de telle sorte que si $A_i \to \alpha$ alors α commence soit par un terminal soit par $A_j, j \geq i$;
On élimine la récursivité directe de A_i en rajoutant les nouvelles règles dans P';
$\to G = (N, T, P', S)$;

2 Quels symboles pour prédire ?

Pour pouvoir prédire correctement, il nous faut étudier le comportement de la grammaire. Pour cela, il nous faudra construire les ensembles :

- $N_\varepsilon = \{A \in N \mid A \overset{*}{\vdash} \varepsilon\}$: l'ensemble des non-terminaux qui peuvent s'effacer (nous savons déjà le calculer),
- $Premiers(\gamma) = \{a \in T \mid \gamma \overset{*}{\vdash} a\alpha\}$ avec $\gamma \in V^*$: l'ensemble des symboles terminaux qui commencent un mot dérivé de la forme sentencielle γ,
- $Suivants(A) = \{a \in T \mid S \overset{*}{\vdash} \alpha A a \beta\}$ avec $A \in N$: l'ensemble des symboles terminaux qui peuvent suivre immédiatement un non terminal A dans une forme sentencielle de G.

Comme il nous faut pouvoir faire cette prédiction même à la fin du mot, on se ramène à une grammaire légèrement modifiée, où on rajoute la règle $S' \to S\$$ avec \$, un symbole qui n'est pas dans le langage décrit et qui représente la fin de mot.

2.1 Calcul de N_ε et $Premiers$

On calcule ces ensembles en étudiant la grammaire G = (N, T, P, S).
On a déjà vu comment calculer N_ε dans le cadre du nettoyage de grammaire. Ici aussi, nous prendrons en pratique l'algorithme à point fixe basé sur la propriété suivante :

$$A \in N_\varepsilon \iff (A \to \alpha) \in P \text{ et } \alpha \in N_\varepsilon^*$$

Pour le calcul de $Premiers(\gamma)$, on utilisera les propriétés :

- $Premiers(\varepsilon) = \emptyset$
- $Premiers(a) = \{a\}$ pour $a \in T$
- $Premiers(X_1..X_n) = Premiers(X_1) \cup \begin{cases} Premiers(X_2..X_n) \text{ si } X_1 \in N_\varepsilon \\ \emptyset \text{ sinon.} \end{cases}$

En pratique, on utilisera un algorithme à point fixe et on pourra calculer N_ε et $Premiers$ en même temps. Pour simplifier le calcul de la fonction $Premiers$ sur les formes sentencielles, on va pré-calculer la fonction sur les non-terminaux.
Pour cela, on remarque, que pour chaque A-production de la forme $A \to X_1...X_n$, on a $Premiers(X_1...X_n) \subset Premiers(A)$.

2.2 Calcul de $Suivants$

Pour le calcul de $Suivants$, on remarque que, si une production est de la forme $X \to \alpha A \beta$, alors :

- $Suivants(A) \supseteq Premiers(\beta)$
- Si $\beta \overset{*}{\vdash} \varepsilon$ alors $Suivants(A) \supseteq Suivants(X)$

La fonction $Suivants$ produit le plus petit ensemble stable pour les 2 relations précédentes. Il peut ici aussi se calculer par point fixe.

2.3 calcul de la table LL_1

La table LL_1 est utilisée pour faire les prédictions valides. Indexée sur $N \times T$, elle contient dans chaque cellule $LL_1(A, a)$ la ou les prédictions valides lorsque les configurations de l'analyse descendante ont un A en sommet de pile et un a en début de mot restant à analyser. Une grammaire sera dite LL_1, lorsque cette table n'aura pas de conflit de prédictions (2 prédictions possibles dans une même cellule).
Pour remplir la table LL_1, on procède production par production :

Calcul de la table LL_1

Entrée:
$G = (N, T, P, S)$; // une grammaire hors contexte
Sortie:
table LL_1 sur $(N \times T)$ de règles ; // l'ensemble des règles que l'on peut prédire
Réalisation:
pour chaque *production* $p_i = A \to \alpha \in P$
  pour chaque $a \in Premiers(\alpha.Suivants(A))$
    Ajouter p_i dans $LL_1(A, a)$;
$\to LL_1$;

Exemple E5.21: Analyse LL_1 des expressions arithmétiques

Nous reprenons la grammaire des expressions arithmétiques sur {a, b}.

E → E+T | T
T → T*F | F
F → (E) | a | b

Cette grammaire est récursive à gauche. Nous la transformons donc en une grammaire équivalente G' sans récursivité gauche. La transformation est ici très simple car le graphe des appels à gauche ne montre pas de récusivité indirecte.
On obtient :

R1 : E → TE′
R2 : E′ → +TE′
R3 : E′ → ε
R4 : T → FT′
R5 : T′ → *FT′
R6 : T′ → ε
R7 : F → (E)
R8 : F → a
R9 : F → b

Pour calculer correctement les Suivants, il nous faut rajouter la règle R0 : $S \to E\$$.
Calculons maintenant *Premiers* (et N_ε) puis *Suivants* :

Premiers

	a	b	+	*	(	)	ε
S	✓	✓			✓		
E	✓	✓			✓		
E′			✓				✓
T	✓	✓			✓		
T′				✓			✓
F	✓	✓			✓		

Suivants

	a	b	+	*	(	)	$
S							
E						✓	✓
E′						✓	✓
T			✓			✓	✓
T′			✓			✓	✓
F			✓	✓		✓	✓

Puis la table LL_1 :

	a	b	+	*	(	)	$
S	p_0	p_0			p_0		
E	p_1	p_1			p_1		
E′			p_2			p_3	p_3
T	p_4	p_4			p_4		
T′			p_6	p_5		p_6	p_6
F	p_8	p_9			p_7		

Il n'y a aucun conflit, la grammaire est donc LL_1. L'analyse descendante (l'automate à pile descendant) peut

donc être guidée par cette table, qui détermine, à chaque étape, en fonction du sommet de pile et du caractère courant, la règle à prédire.

Dans l'automate à pile descendant correspondant à notre grammaire G′, nous aurons donc la trace d'acceptation suivante pour $u = a + b * a\$$, guidée par la table LL_1 :

état	**entrée**	**pile**	**commentaire**
q_0	a+b*a$	S	configuration initiale
q_0	a+b*a$	E$	p_0
q_0	a+b*a$	TE′$	p_1
q_0	a+b*a$	FT′E′$	p_4
q_0	a+b*a$	aT′E′$	p_8
q_0	+b*a$	T′E′$	lecture
q_0	+b*a$	E′$	p_6
q_0	+b*a$	+TE′$	p_2
q_0	b*a$	TE′$	lecture
q_0	b*a$	FT′E′$	p_4
q_0	b*a$	bT′E′$	p_9
q_0	*a$	T′E′$	lecture
q_0	*a$	*FT′E′$	p_5
q_0	a$	FT′E′$	lecture
q_0	a$	aT′E′$	p_8
q_0	$	T′E′$	lecture
q_0	$	E′$	p_6
q_0	$	$	p_3
q_0	ε	ε	lecture (et configuration finale)

2.4 Que faire quand une grammaire n'est pas LL_1

Il peut y avoir diverses raisons pour qu'une grammaire ne soit pas LL_1. Elle peut :

- contenir une récurrence à gauche. Dans ce cas, on peut tenter d'éliminer cette récurrence à gauche.
- être ambiguë. En effet, si 2 arbres sont possibles pour un mot, il y aura forcément deux prédictions possibles à une certaine étape du calcul.
- contenir des A-productions ayant même préfixe. Dans ce cas, on peut tenter de factoriser ces préfixes et de construire une nouvelle grammaire.

Néanmoins, il n'existe pas de garantie que l'on puisse se ramener au cas LL_1.

On peut aussi étudier l'analyse LL_2, qui se fait selon les mêmes principes, si ce n'est qu'on cherche à savoir quels sont les $Premiers_2$ (les mots de longueur 2 qui peuvent commencer un mot dérivé d'une forme sentencielle), les $Suivants_2$ (les mots de longueur 2 qui peuvent suivre un non terminal) et une table LL_2 sur $(N \times T^2)$.

Hélas, il existe des langages dont on sait qu'ils n'admettent pas de grammaire LL_k (pour tout k). Nous venons donc de voir une méthode particulièrement rapide d'analyse, mais qui ne permet pas l'analyse de tous les langages hors-contexte.